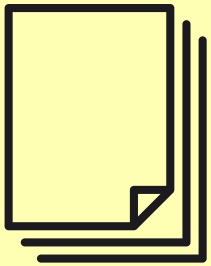
Starter

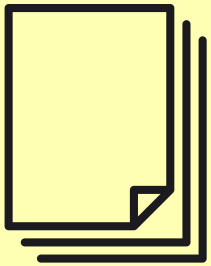


1. What is the difference between an object and an object reference?

2. Give an example of C# code that creates a new object.

3. How does the Call Stack grow and shrink in size during run time?

Starter



1. What is the difference between an object and an object reference?

An object stores actual data in the heap, whereas an object reference is a memory reference value stored in a variable, which points to an object stored in the heap.

2. Give an example of C# code that creates a new object.

```
List<int> nums = new List<int>();
```

3. How does the Call Stack grow and shrink in size during run time?

Grows with a stack frame when a new subroutine is called. Shrinks when a subroutine ends

Starter 2

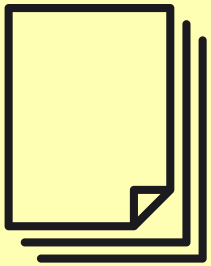
1. What are the **properties** of a duck?
   - Not just physical appearance
2. How could you represent this in a C# structure?
3. What are the **behaviours** of a duck? What can it do?

EXT: What are the differences between a **structure** and a **class**

Object-Oriented Programming

Keywords: Programming paradigm; Object; Class; Instantiation; Attribute; Property; Member variable; Method; Encapsulation;

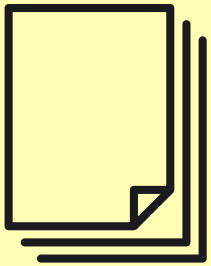
Objects



Objects in the real world have:

- An identity
- Attributes (measurable things)
- Behaviours (actions it can take)

Objects



Objects in the real world have:

- An identity
- Attributes (measurable things)
- Behaviours (actions it can take)

Objects in a program will have the same!

Classes

A class can be thought of as a blueprint for a related group of objects (eg. ducks!)

It exists in the code written by the programmer.

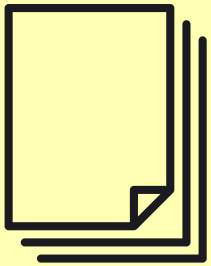
It needs to specify:

- The identifier for the class / group
- Properties/Attributes to store
- Methods/Behaviours to call

EG:

```
class Duck {
ConsoleColor colour;
double Weight, QuackVolume;
int Age;
public void Swim() {...}
}
```

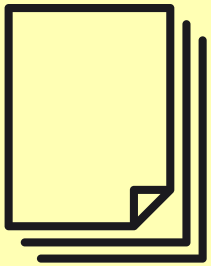
Encapsulation



Encapsulation is:

1. **combining properties and methods** together into an object

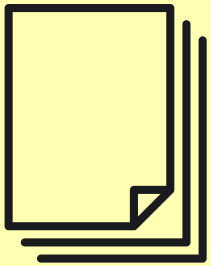
Encapsulation



Encapsulation is:

1. **combining properties and methods** together into an object
2. being able to **restrict access** to an object's state/behaviour.

Encapsulation



Encapsulation is:

1. **combining properties and methods** together into an object
2. being able to **restrict access** to an object's state/behaviour.

Like a Pirate chest!

1. You want to be able to put things **in** the chest.
2. You want to be able to **lock** the chest, sometimes.

But whyyyyyyy?

Imagine that you're making a game, and you need enemies to move around.

But whyyyyyyy?

Imagine that you're making a game, and you need enemies to move around.

Parallel arrays let you track all of their attributes, and you have functions to take care of their behaviours.

But whyyyyyyy?

Imagine that you're making a game, and you need enemies to move around.

Parallel arrays let you track all of their attributes, and you have functions to take care of their behaviours.

A new attribute needs to be added to the game, so you implement a `struct` rather than a new array, as you've got a few now...

But whyyyyyyy?

Imagine that you're making a game, and you need enemies to move around.

Parallel arrays let you track all of their attributes, and you have functions to take care of their behaviours.

A new attribute needs to be added to the game, so you implement a `struct` rather than a new array, as you've got a few now...

You've got several functions that use `switch` on a property of that enemy, but when you want to add a new type, you have to edit every one of those functions!

Option 1:

```
static Enemy[] allEnemies;

static void enemyAction(Enemy e){
  switch(e.type){
    ...

static void removeEnemy(Enemy e){
  switch(e.health){
    ...

static void ...
```

functions!

d you need enemies to move

attributes, and you have functions

he game, so you implement a

ve got a few now...

witch on a property of that enemy,

you have to edit every one of those

Option 1:

```
static Enemy[] allEnemies;

static void enemyAction(Enemy e
  switch(e.type){
    ...

static void removeEnemy(Enemy e
  switch(e.health){
    ...

static void ...
```

functions!

Option 2:

```
static Enemy[] allEnemies;

class Enemy {

  void Act() {
    ...
  }
  void Damage() {
    ...
  }
  void Move() {
    ...
```

ve

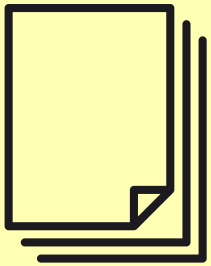
ctions

a

enemy,

f those

Objects vs Classes



A **class** is the blueprint for the object. It's a template.

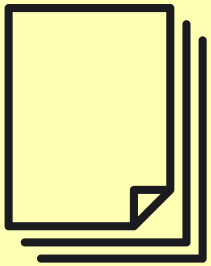
An **object** is an isntance of that class.

Instantiation is the process of creating an object from the class.

Warning!

We are going outside of the class `Program`

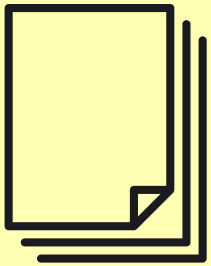
Classes in C#



These will look like structures, so I'll put them both here for comparison. (Remember, comparison is a useful memory tool!)

```
public struct User {
public string userName;
public string passHash;
public bool isAdmin; }

class Program {
  static void logInUser(User u) {
    ... }
  static void showPage(User u) {
    ...
}
```

Classes in C#

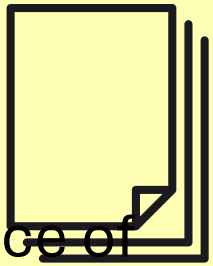


These will look like structures, so I'll put them both here for comparison. (Remember, comparison is a useful memory tool!)

```
public struct User {
public string userName;
public string passHash;
public bool isAdmin; }

class Program {
  static void logInUser(User u) {
    ... }
  static void showPage(User u) {
    ...
}
```

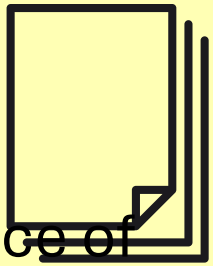
```
public class User {
  public string userName, passHash;
  public bool isAdmin;

  public User(string inName, string pass){
    ...
  }
  bool validateLogin(string inPass) {
    ... }
  static void displayPage() {
    ... }
}
```

Instantiation



Once we've created a class, we need to be able to create an instance of it within our program!

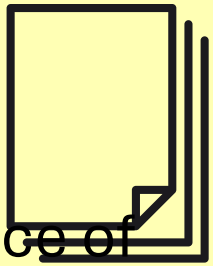
Instantiation



Once we've created a class, we need to be able to create an instance of it within our program!

```
static void Main(string[] args) {
  User mainAccount = new User();

  Console.WriteLine("Please log in:");
  mainAccount.validateLogin(Console.ReadLine());
}
```

Instantiation



Once we've created a class, we need to be able to create an instance of it within our program!

```
static void Main(string[] args) {
  User mainAccount = new User();

  Console.WriteLine("Please log in:");
  mainAccount.validateLogin(Console.ReadLine());
}
```

The term `new User();` calls the **constructor** of the class. If one hasn't been programmed, C#'s compiler will use the **default constructor**. If one has been programmed, the default constructor does not get created.

Practice Time!

On BPCompSci.com: **W125 - C# Using classes and objects**

```
public class Account
{
    public int balance;
    public int overdraftLimit;
    public void makeDeposit(int amount)
    {
        balance += amount;
    }
    public void MakeWithdrawal(int amount)
    {
        balance -= amount;
    }
}
```

```
static void Main(string[] args)
{
    Account myAccount = new Account();
    myAccount.MakeWithdrawal(1337);
}
```

Practice Time!

On BPCompSci.com: **W125 - C# Using classes and objects**

```
public class Account
{
    public int balance;
    public int overdraftLimit;
    public void makeDeposit(int amount)
    {
        balance += amount;
    }
    public void MakeWithdrawal(int amount)
    {
        balance -= amount;
    }
}
```

```
static void Main(string[] args)
{
    Account myAccount = new Account();
    myAccount.MakeWithdrawal(1337);
}
```

EXT: W126 - C# Creating custom classes